Two-Factor Authentication by OpenOTP

This article is a description of how to use OpenOTP, by rcdev, to set up a complete environment for two-factor authentication on various servers and for various applications. Readers should have a knowledge of Linux, OpenLDAP, and RADIUS.

Before I jump into the topic, let me explain what OpenOTP is, and why we may need it. Password-based authentication has been around for a long time—perhaps even during the days of the Colosseum!—but is a debatable method, because our secrets are not as secret as we would like them to be. Social engineering, shoulder-surfing (when you log in), eavesdropping, etc, can capture your secret/password. Besides, some people use weak passwords based on easily-found personal data, like ex-girlfriends' or spouse's names, dates of birth, etc. This is where two-factor authentication comes in, helping to keep your network secure.

Two-factor authentication is a mechanism where more than one thing is required to authenticate a user. The two components of two-factor authentication are:

- Something you know (e.g., password/PIN, etc)
- Something you have (a token, cell phone, etc)

The 'two-factor' or 'strong authentication' mechanism is based on one-time passwords (OTP). In this mechanism, instead of authenticating with a simple password, every user will use an OTP, generated from a device (hardware token), or by an application, perhaps on their cell phone. This is, no doubt, more secure than a static password, because the attacker needs to know both the PIN, and possess the token/device you are using. If the attackers try eavesdropping on your login session too, they won't succeed, since a one-time password is valid for only one session. Once an OTP is generated, it cannot be re-used.

One of the biggest issues with two-factor authentication is expense, because special-purpose hardware tokens have a cost—and also, the authentication server has licence costs. Besides, the algorithm used to produce the codes is only known to the company—i.e., it is proprietary. The solution I'm proposing is not fully open source, but is based on open standards and Linux. It supports hardware tokens.

In my current assignment, I had to come up with a solution that would support OTP for applications like RADIUS, SSH, Apache, and some other Web services as well.

Out of personal interest, I had in the past, used open source tools like OPIE (One-time Passwords In Everything), Mobile-OTP, Google Authenticator, etc, but the problem with these is that the secret and pass-phrase are stored in plain text files on the desktop file-system. There are no schema available to integrate these applications with OpenLDAP or any directory service.

I also had to invest a lot of time and effort on scalability, service uptime, and plug-ins for different applications. I was looking for something that could store everything centralised in a directory server like OpenLDAP, and also was very